

# WORK ALLOCATION DOCUMENT

## PhishShield<br/>A Machine Learning-Based Web Application for Phishing URL Detection

| Document Item     | Details                                   |
|-------------------|-------------------------------------------|
| Team Size         | 4 Members                                 |
| Database          | MySQL                                     |
| UI/UX             | Web interface and user experience         |
| Machine Learning  | Phishing URL classification               |
| Work Distribution | 2 Database + 1 UI/UX + 1 Machine Learning |

### 1. Introduction

This document defines the responsibilities, deliverables and coordination activities of the four members working on PhishShield. The project is divided into database development, UI/UX design and machine learning. Two members handle the database module, one handles UI/UX, and one handles machine learning.

### 2. Objectives of Work Allocation

- Divide the project into clearly defined technical modules.
- Assign two members to MySQL database design and implementation.
- Assign one member to UI/UX design and user interaction.
- Assign one member to dataset preparation and machine learning.
- Define measurable deliverables for each member.
- Establish integration points between all modules.

### 3. Team Structure

| Member   | Role                             | Primary Area                                         |
|----------|----------------------------------|------------------------------------------------------|
| Member 1 | Database Developer – Design      | MySQL schema, tables, constraints and data integrity |
| Member 2 | Database Developer – Integration | MySQL connectivity, CRUD operations and scan records |
| Member 3 | UI/UX Designer                   | Wireframes, user flow and interface design           |
| Member 4 | Machine Learning Engineer        | Dataset, features, model training and evaluation     |

# DATABASE WORK ALLOCATION

## 4. Database Module Overview

The database module is divided between two members. Member 1 focuses on logical database design and MySQL structure. Member 2 focuses on backend integration and database operations. Both members coordinate to ensure the schema and application queries remain consistent.

## 5. Member 1 – Database Design and MySQL Structure

| Area            | Responsibilities                                                      |
|-----------------|-----------------------------------------------------------------------|
| Database Design | Identify entities, attributes and relationships.                      |
| Schema Design   | Create the MySQL database and required tables.                        |
| Table Structure | Define scan_id, url, prediction, recommendation and scan_time.        |
| Constraints     | Implement primary keys, NOT NULL constraints and suitable data types. |
| Normalization   | Maintain a clean relational design with minimal duplication.          |
| Documentation   | Prepare schema, ER/logical structure and database documentation.      |

## 6. Member 1 – Deliverables

- Final MySQL schema.
- Table definitions and constraints.
- ER/logical database structure.
- Sample development records.
- Database design documentation.

## 7. Member 2 – Database Integration and Operations

| Area                | Responsibilities                                                   |
|---------------------|--------------------------------------------------------------------|
| MySQL Connectivity  | Connect the Python Flask backend to MySQL.                         |
| CRUD Operations     | Implement required insert, retrieve, update and delete operations. |
| Scan Storage        | Store URL, prediction, recommendation and scan time.               |
| Backend Integration | Ensure API and MySQL communicate correctly.                        |
| Validation          | Handle invalid or incomplete data before insertion.                |
| Testing             | Test queries, connection failures and stored scan records.         |

## 8. Member 2 – Deliverables

- Working MySQL connection.
- Database query/API integration.
- Scan record insertion and retrieval.
- Database test cases.
- Integration documentation.

# UI/UX AND MACHINE LEARNING ALLOCATION

## 9. Member 3 – UI/UX Designer

The UI/UX member designs how users interact with PhishShield. The interface should make entering a URL and understanding the detection result simple, clear and accessible. The design must match the prediction and response information supplied by the backend.

| Area              | Responsibilities                                                   |
|-------------------|--------------------------------------------------------------------|
| User Flow         | Define the steps from URL entry to result.                         |
| Wireframes        | Design URL checker, loading, result and error states.              |
| Visual Design     | Define typography, spacing, colors, buttons and result indicators. |
| Responsive Design | Plan desktop, tablet and mobile layouts.                           |
| Usability         | Make predictions and security recommendations easy to understand.  |
| Handoff           | Provide specifications for frontend implementation.                |

## 10. Member 3 – Deliverables

- UI/UX design document.
- Wireframes and user flow.
- URL input and prediction-result specifications.
- Responsive design guidelines.
- Usability and accessibility notes.

## 11. Member 4 – Machine Learning Engineer

The ML member develops the intelligence behind phishing URL classification. This includes preparing labeled data, extracting URL-based features, training candidate models, evaluating performance and preparing the selected model for backend integration.

| Area                | Responsibilities                                                                                    |
|---------------------|-----------------------------------------------------------------------------------------------------|
| Dataset             | Collect and prepare phishing and legitimate URL data.                                               |
| Preprocessing       | Clean data, remove duplicates and verify labels.                                                    |
| Feature Engineering | Extract URL length, dots, hyphens, digits, suspicious symbols/keywords and other selected features. |
| Model Development   | Train and compare suitable classification algorithms.                                               |
| Evaluation          | Measure accuracy, precision, recall, F1-score and confusion matrix.                                 |
| Model Export        | Prepare the selected model for deployment.                                                          |
| Documentation       | Document dataset, features, algorithm selection and results.                                        |

## 12. Member 4 – Deliverables

- Prepared dataset.
- Feature extraction module.
- Trained models and comparison results.
- Selected deployment model.
- Evaluation report.
- Model integration requirements.

# MODULE INTEGRATION AND COLLABORATION

## 13. Integration Between Members

The three technical areas are interdependent. Database members provide storage and retrieval, the UI/UX member defines user interaction and presentation, and the ML member provides prediction output consumed by the backend and displayed through the interface.

| From            | To                  | Integration Requirement                                              |
|-----------------|---------------------|----------------------------------------------------------------------|
| UI/UX           | Backend             | Frontend needs agreed API fields and response format.                |
| ML              | Backend             | Model input features and prediction output need a defined interface. |
| Backend         | MySQL               | Prediction and scan information must use the agreed schema.          |
| Database Design | Backend Integration | Table names, fields and constraints must match queries.              |
| All Members     | Testing             | Integrated workflow must be tested end-to-end.                       |

## 14. Overall Application Workflow

| Step | Module          | Activity                                                  |
|------|-----------------|-----------------------------------------------------------|
| 1    | UI/UX           | User enters URL.                                          |
| 2    | Backend + DB    | Request is validated.                                     |
| 3    | ML              | URL features are extracted.                               |
| 4    | ML              | Model returns Legitimate or Phishing.                     |
| 5    | MySQL           | URL, prediction, recommendation and scan time are stored. |
| 6    | Backend + UI/UX | Result and recommendation are displayed.                  |

## 15. Collaboration Requirements

- Agree on database field names before backend implementation.
- Agree on prediction labels used by ML and UI.
- Define API request and response structure before integration.
- Document changes affecting another module.
- Perform integration checks after major milestones.
- Use version control for code and documentation.

# TIMELINE AND RESPONSIBILITY MATRIX

## 16. Suggested Development Timeline

| Week | M1 DB Design          | M2 DB Integration        | M3 UI/UX          | M4 ML               |
|------|-----------------------|--------------------------|-------------------|---------------------|
| 1    | Requirements + schema | Backend requirements     | User flow         | Dataset research    |
| 2    | MySQL schema          | MySQL connection         | Wireframes        | Dataset collection  |
| 3    | ER/documentation      | CRUD queries             | Visual design     | Preprocessing       |
| 4    | Schema testing        | Backend integration      | Responsive specs  | Feature engineering |
| 5    | DB refinement         | API testing              | Frontend support  | Model training      |
| 6    | Integration support   | Full backend integration | UI refinement     | Model evaluation    |
| 7    | DB testing            | End-to-end testing       | Usability testing | Model integration   |
| 8    | Final docs            | Final integration        | Final UI          | Final ML docs       |

## 17. Responsibility Matrix

| Task                | M1      | M2      | M3      | M4      |
|---------------------|---------|---------|---------|---------|
| Database schema     | Lead    | Support | —       | Consult |
| MySQL integration   | Support | Lead    | —       | —       |
| Wireframes / UI     | —       | Consult | Lead    | Consult |
| Dataset             | —       | —       | —       | Lead    |
| Feature engineering | —       | —       | —       | Lead    |
| Model training      | —       | —       | Consult | Lead    |
| API integration     | Support | Lead    | Support | Support |
| System testing      | Support | Lead    | Lead    | Lead    |
| Final documentation | Support | Support | Lead    | Lead    |

# QUALITY ASSURANCE AND CONCLUSION

## 18. Review and Quality Assurance

Each member reviews their own module before integration. Cross-module reviews are necessary because MySQL fields, API responses, ML predictions and UI elements must use compatible definitions. The final system should be tested from URL submission through prediction and database storage.

| Review Area   | Responsible Members | Focus                                        |
|---------------|---------------------|----------------------------------------------|
| MySQL Schema  | M1 + M2             | Fields, constraints, queries and integrity.  |
| UI/UX         | M3 + Team           | Usability, responsiveness and clarity.       |
| ML Model      | M4 + Team           | Performance, features and prediction output. |
| Integration   | All                 | End-to-end URL-to-result-to-MySQL workflow.  |
| Documentation | All                 | Completeness and consistency.                |

## 19. Final Work Allocation Summary

| Member   | Role                             | Main Deliverable                                     |
|----------|----------------------------------|------------------------------------------------------|
| Member 1 | Database Developer – Design      | MySQL schema, tables, constraints and logical design |
| Member 2 | Database Developer – Integration | MySQL connectivity, CRUD and backend integration     |
| Member 3 | UI/UX Designer                   | Wireframes, user flow and interface specifications   |
| Member 4 | Machine Learning Engineer        | Dataset, features, trained model and evaluation      |

## 20. Conclusion

The work allocation divides PhishShield into clearly owned modules while preserving the collaboration required for integration. Two members jointly handle the MySQL database—one focusing on database design and the other on backend integration. The UI/UX member focuses on the user-facing experience, while the ML member develops the phishing URL classification system. This structure provides clear responsibilities, measurable deliverables and defined integration points for completing the project as a four-member team.